



MEMRISTOR-BASED INTEGRATION OF HIERARCHICAL TEMPORAL MEMORY FOR ENERGY- EFFICIENT NEUROMORPHIC COMPUTING

Abstract:

This dissertation explores the integration of memristor technology with Hierarchical Temporal Memory (HTM) to create an energy-efficient and biologically inspired computing framework. The limitations of conventional HTM implementations, such as inefficiency in hardware and scalability, are addressed by introducing memristors as a compact, non-volatile, and low-energy hardware solution. The research develops a novel memristor-HTM framework, evaluates its algorithmic and hardware performance, and benchmarks it against state-of-the-art models like LSTMs. Results demonstrate improvements in classification accuracy, sparsity, entropy, energy consumption, and scalability, positioning this work as a significant step toward brain-inspired computing systems.

James T. McAlinden

mcAlindenjames02@gmail.com / 40298160@ads.qub.ac.uk

Introduction

Context and Motivation:

The Hierarchical Temporal Memory (HTM) is a biologically inspired computational framework that attempts to replicate the structural and functional properties of the neocortex. At the core of HTM are sparse distributed representations (SDRs), which enable the system to efficiently encode and learn spatial and temporal patterns. Such representations have proven useful in applications ranging from anomaly detection and pattern recognition to time-series forecasting. However, most HTM implementations rely on traditional CMOS-based circuits, leading to scalability challenges, significant power consumption, and large area footprints when simulating large numbers of synapses or neurons.

Memristors have emerged as a promising candidate for addressing the hardware inefficiencies of HTM. These non-volatile memory elements, which store information in their resistance states, can emulate synaptic plasticity in a highly compact and energy-efficient way. By integrating memristors into the HTM pipeline, we can preserve HTM's algorithmic strengths like robust SDR encoding while mitigating the prohibitive energy and circuit-area costs of conventional CMOS-only approaches.

Research Questions:

Building upon these motivations and code features, this dissertation addresses the following key questions:

- Can memristors effectively reduce the energy and hardware complexity in HTM implementations compared to traditional CMOS approaches?
- How does integrating memristive synapses with HTM affect classification metrics (accuracy), representational properties (sparsity, entropy), and overall robustness?
- Does the proposed memristor-based HTM system maintain or improve performance relative to other state-of-the-art hardware and software-based solutions?

Aim and Objectives

Aim: To design, implement, and rigorously evaluate a **memristive HTM** system that preserves HTM's core algorithmic advantages—particularly its use of SDRs—while reducing the energy, area, and complexity overhead typically associated with large-scale HTM deployments.

Objectives:

- **Objective 1:** Develop a fully functional HTM spatial pooler in MATLAB that incorporates memristor-model synaptic updates through adapted permanence values.
- **Objective 2:** Implement and analyze advanced adaptive mechanisms (dynamic kWTA, adjustable thresholding, noise injection) to ensure stable learning.
- **Objective 3:** Quantitatively evaluate classification performance, sparsity, and entropy on standard datasets (e.g., MNIST, CIFAR-10).
- **Objective 4:** Benchmark the energy efficiency of the proposed memristor-based system against CMOS-based designs and measure its practical viability.

Contributions Added:

- **Memristor-Based HTM Framework**
 - Developed a **novel codebase** integrating memristor-like weight updates into the HTM Spatial Pooler, aligning device-level physics with synaptic plasticity.
 - Incorporated *memristor endurance* and *write-energy constraints* to move beyond idealized permanence updates.
- **Adaptive and Energy-Aware Design**
 - Implemented a **homeostatic PI controller** (`adjust_density.m`) that dynamically adjusts overall sparsity and a **momentum-driven synaptic scaling** mechanism (`adjust_synaptic_factors.m`), enhancing stability and preventing runaway activity.
 - Monitored cumulative **memristor energy usage** in real time, scaling synaptic update rates to respect endurance limits and power budgets.
- **Advanced k-WTA and Entropy Regulation**
 - Introduced **local adaptive inhibition** in `apply_kwta.m`, allowing the inhibition radius to expand or contract based on local variance and global correlations.
 - Used *fallback activation* and *stochastic exploration* to ensure minimal active columns even under harsh inhibition.
 - Employed **entropy-based resets** to counteract representational collapse and keep column activations diverse over time.
- **Performance and Metric Evaluation**
 - Measured classification accuracy, SDR sparsity, and entropy using a variety of **adaptive thresholds** and noise injection methods to assess robustness.
 - Leveraged **Bayesian hyperparameter optimization** (`optimize_hyperparameters.m`) for systematic tuning of parameters (e.g., base density, LTP/LTD rates, decay factors) instead of ad-hoc manual selection.
- **Comparative Circuit-Level Insights**
 - Conducted preliminary **circuit-level benchmarks** between memristor-based HTM updates and conventional CMOS designs, highlighting potential improvements in energy consumption, scalability, and biological plausibility.
- **PCA-Based Weight Initialization**
 - Used PCA-generated matrices in `initialize_permanence.m` to give columns a more biologically plausible (and empirically beneficial) starting point compared to a purely random initialization
 - This speeds up convergence and mimics the idea of preconfigured receptive fields.

Literature Review

Hierarchical Temporal Memory (HTM):

Overview of HTM:

HTM, inspired by the structure and functionality of the neocortex, models human intelligence through a hierarchy of spatial and temporal memory layers. Its key components include:

- Spatial Pooler (SP): Generates sparse distributed representations (SDRs) by selecting a subset of active columns, ensuring efficient and robust encoding of input patterns.
- Temporal Pooler (TP): Encodes sequential patterns, enabling the system to learn and predict temporal dependencies in data.
- Sparse Distributed Representations (SDRs): Binary representations where a small, fixed percentage of elements are active, offering robustness to noise and efficiency in memory use.

Biological Inspiration: Mimics how neurons in the brain process data in a hierarchical and spatiotemporal manner.

Challenges in Current Implementations:

Hardware Scalability Issues: Conventional HTM frameworks rely heavily on software implementations that are computationally expensive and challenging to scale for real-time applications.

Energy Inefficiency: Existing CMOS-based implementations of HTM struggle with high power consumption, particularly when simulating large-scale neuron and synapse networks.

Data Encoding Bottlenecks: Generating and processing SDRs at high resolution requires significant computational resources.

Memristors in Neuromorphic Computing:

Theoretical Background:

Memristors are non-volatile memory devices that store information as resistance levels, emulating synaptic plasticity in biological systems.

Synaptic Behavior: Memristors naturally support the weight update mechanisms required for learning in neuromorphic computing. Their analog behavior aligns closely with biological synapses, allowing efficient weight adjustments and memory retention.

Applications in Neuromorphic Hardware:

Memristors have been successfully used in:

- Spiking Neural Networks (SNNs) for real-time processing.
- Energy-efficient deep learning accelerators.
- Memory-centric hardware for inference tasks.

For instance, the "CMOS K-Winners-Take-All Circuit" demonstrates a hybrid memristor-CMOS approach for reducing computational complexity in competitive learning integrations with Machine Learning Models:

While several studies have incorporated memristors into artificial neural networks (ANNs), their integration with HTM frameworks remains limited. Previous works highlight:

- **Energy Advantages:** Memristors reduce power consumption in crossbar array implementations compared to CMOS-only circuits.
- **Lack of HTM-Specific Research:** The research primarily focuses on neural networks rather than HTM, leaving a gap in exploring memristors for hierarchical and temporal learning.

Research Gaps:

Hardware Inefficiencies in HTM:

Although HTM has shown promise in spatiotemporal learning tasks (e.g., anomaly detection and pattern recognition), its reliance on traditional hardware limits real-world applications.

The spatial and temporal pooler operations require efficient mechanisms for winner selection, sparse representation, and weight updates, which are computationally taxing when implemented on software or CMOS circuits.

Limited Research on HTM-Memristor Integration:

While memristors have proven their value in other neuromorphic computing paradigms, their potential to optimize HTM frameworks remains underexplored.

Few studies address the design of memristor-based spatial poolers or k-Winners-Take-All (kWTA) circuits for HTM. For example:

- The "Memristor-Based HTM Spatial Pooler With On-Device Learning for Pattern Recognition" highlights a pathway but focuses on static pattern recognition rather than dynamic spatiotemporal tasks .
- The "CMRs-Take-All Circuit With $O(N)$ Complexity" demonstrates algorithmic efficiency, but integrating such designs with HTM remains a challenge .

These gaps suggest the following directions for research:

- **Robust Integration:** Developing a robust integration of HTM with memristor-based hardware to achieve improved energy efficiency and scalability.
- **Comprehensive Benchmarking:** Benchmarking memristor-HTM systems against state-of-the-art algorithms like LSTMs or transformer models to establish computational and energy efficiency benefits.
- **Dynamic Applications:** Extending research to address dynamic spatiotemporal tasks, leveraging memristor technologies to enhance HTM's adaptability in such scenarios

Methodology

HTM Framework Design:

1. Spatial Pooler:

- Implements k-Winners-Take-All (kWTA) mechanism to select active columns.
- Optimizes sparsity and entropy for efficient SDR generation.

2. Temporal Pooler:

- Learns temporal patterns in data, enhancing the system's ability to predict future inputs.

Memristor Integration:

- Memristors replace traditional synapses in HTM, where resistance levels represent synaptic weights.
- Advantages include:
 - Non-volatility for weight retention.
 - Energy-efficient updates during learning.
 - Compact circuits enabling scalability.

Performance Metrics:

- **Algorithmic:** Classification accuracy, sparsity (percentage of active columns), entropy (information retention).
- **Hardware:** Energy consumption, circuit area, and efficiency.

Experimental Setup:

- Datasets: MNIST for digit recognition and CIFAR-10 for image classification.
- Hyperparameter tuning for kWTA thresholds, learning rates, and sparsity levels.
- Simulation environment: MATLAB and Simulink.

Experiments and Results

Dataset: MNIST and CIFAR-10.

Metrics: Accuracy, Sparsity, Entropy, Energy Consumption.

Validation flow: k-fold cross-validation + Bayesian hyperparameter tuning.

Energy/performance plots using metrics_analysis.m.

Ablation Studies:

- Impact of removing key components (e.g., kWTA, synaptic updates) on overall performance.
- Validation of each component's contribution through performance degradation analysis.

Comparisons:

LSTM (accuracy vs. energy).

CMOS kWTA vs. memristor model (complexity)

Discussion

Interpret metric trade-offs: entropy vs. sparsity.

Highlight improvements in write cycles and duty cycles.

Insights into practical viability of memristive HTM.

Address limitations: simulated memristors vs. real hardware.

Conclusion

Summary:

- Successfully integrated memristors into HTM, addressing key hardware inefficiencies.
- Demonstrated improvements in algorithmic and hardware performance.

Implications:

- Advances brain-inspired computing by combining algorithmic innovation with efficient hardware.
- Opens pathways for scalable and energy-efficient neuromorphic systems.

Future Work:

- Investigate hybrid models (e.g., combining HTM and LSTM).
- Extend memristor-based HTM to more complex datasets and real-world tasks.

References

Appendices

1. **Additional Results:**

- Hyperparameter tuning logs.
- Supplemental plots of entropy and sparsity trends.

2. **Code Snippets:**

- Key algorithms, kWTA, entropy calculation, synaptic updates.

3. **References:**

- citations for HTM, memristor technology, comparative studies.

Graphs:**1. Diagrams:**

- HTM architecture before and after memristor integration.
- Circuit-level design showing memristor-based synapses.

2. Tables:

- Algorithmic metrics: accuracy, sparsity, entropy.
- Hardware metrics: energy consumption, area efficiency.

3. Graphs:

- Entropy and sparsity trends across training epochs.
- Classification accuracy comparisons.